Table of Contents

Sr. No.	Title	Page No.
1	1. Introduction and system overview	2
2	2. Coverage at a glance	4
3	3. Criterion 1: Messaging architecture and context sharing	5
4	4. Criterion 2: Tool integration using MCP	7
5	5. Criterion 3: Structured output with Pydantic	9
6	6. Criterion 4: Agentic design patterns	11
7	7. Criterion 5: Task monitoring and logging with callbacks	13
8	8. Criterion 6: Agent-to-agent communication with A2A	15
9	9. Criterion 7: Agentic frameworks: CrewAI and ADK only	17
10	10. Criterion 8: Guardrails and security	19
11	11. Criterion 9: Observability: tracing, auditability and provenance	21
12	12. Criterion 10: End-to-end evaluation, cost and running time	23
13	13. Criterion 11: Cloud deployment	24
14	14. Evaluation results	25
15	15. Project timeline and progress against the plan	27
16	16. Verification and testing	28
17	17. Conclusion and remaining work	29
18	Appendix A. Repository map	30
19	Appendix B. How to reproduce the results	30

1. Introduction and system overview

1.1 Purpose of this report

The course specifies eleven minimum criteria that every project must meet, so that projects share a common baseline. This report goes through each criterion in turn. For every one it states the requirement, what we built to satisfy it, how it works in the code, and where it can be inspected and verified. It is the long form of the Coverage Matrix presented in the project slides.

1.2 The project in brief

Modern CI/CD pipelines produce build logs, test results, system metrics and source changes. When a pipeline fails, an engineer must open all of them and correlate them by hand. JenkinsGuardian AI automates that investigation with a team of cooperating agents, each specialised in one kind of evidence, and keeps a human in control of any change that results.

When a Jenkins build fails, a webhook creates an incident. Three investigation agents then work in parallel and write their findings to a shared Blackboard: the **Jenkins agent** reads the console log, stages and test report; the **Metrics agent** reads Prometheus time series around the failure; the **Code agent** reads the failing commit, pull requests and diffs. A **Coordinator** agent weighs the three independent conclusions, detects conflicts, and produces one incident report. If the report is strong enough, the Code agent proposes a minimal patch, which passes an OPA policy gate and a named human reviewer before a draft pull request is opened. Jenkins then re-runs the build, and a failure sends the incident back to investigation.

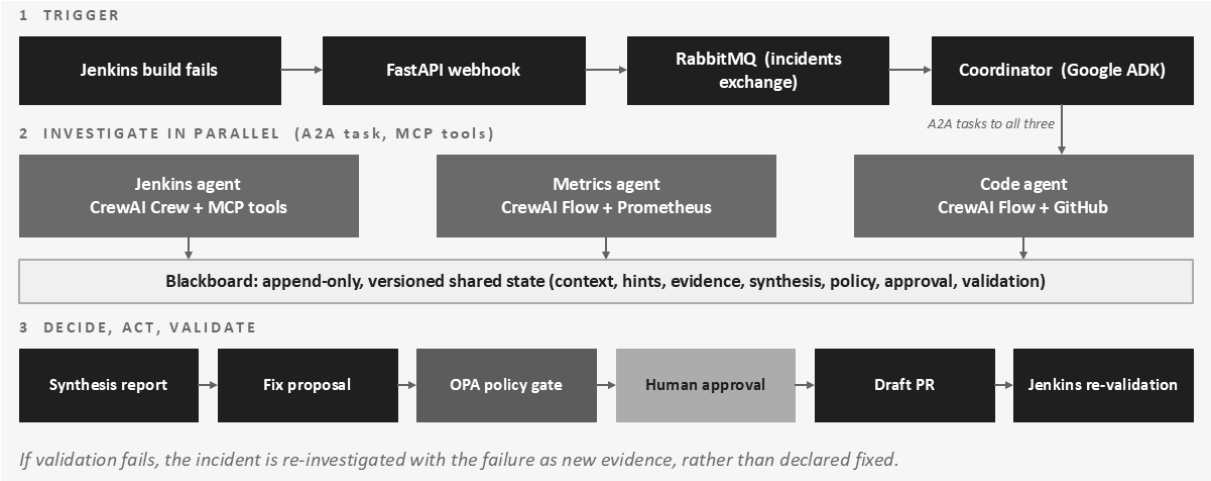


Figure 1. System architecture: trigger, parallel investigation over a shared Blackboard, and the guarded fix loop.

1.3 Components

Component	Technology	Responsibility
Jenkins agent	CrewAI Crew, Gemini through a shared LLM client, Jenkins and analytics MCP tools	Reads builds, classifies the failure with a ReAct loop, cites evidence
Metrics agent	CrewAI Flow, Prometheus, robust statistics	Detects anomalies against a baseline and correlates metrics
Code agent	CrewAI Flow, GitHub API	Finds whether a code change explains the failure; builds minimal fixes
Coordinator	Google ADK, A2A	Dispatches agents, synthesises one report, drives the remediation gate
Blackboard	Append-only versioned store	Shared state between all components
Event bus	RabbitMQ topic exchange	Incident, approval and validation events
Policy gate	OPA / Rego	Decides whether a fix may exist and whether a PR may open
Backend and console	FastAPI, PostgreSQL, React	Webhook, approvals, audit trail, web console with roles
Monitoring	Prometheus, Grafana	Application, Jenkins and agent-operations dashboards, alerts

2. Coverage at a glance

Nine of the eleven criteria are covered. The remaining two, the large end-to-end cost and latency study (criterion 10) and cloud deployment (criterion 11), are planned for the final week; Section 17 sets out the plan.

#	Criterion	Status	How it is met
1	Messaging architecture and context sharing (Blackboard)	**Covered**	Append-only, versioned Blackboard with write permissions per author; RabbitMQ events and A2A triggers alongside
2	Tool integration using MCP (at least two tools)	**Covered**	Jenkins MCP (7 tools) and Analytics MCP (3 tools) in the running stack; Prometheus MCP server
3	Structured output with Pydantic	**Covered**	Evidence and IncidentReport models with a generated canonical JSON schema; Markdown rendering
4	Agentic design patterns	**Covered**	Hierarchical Coordinator, parallel dispatch, sequential flows, ReAct and plan-and-execute prompts
5	Task monitoring with callbacks	**Covered**	CrewAI step and task callbacks, audit events for every agent, Prometheus metrics, run-duration heat map
6	Agent-to-agent communication with A2A	**Covered**	Google ADK Coordinator and CrewAI agents exchange tasks and notifications over A2A
7	Only CrewAI and ADK	**Covered**	CrewAI for the three investigators, Google ADK for the Coordinator
8	Guardrails and security	**Covered**	Injection and toxicity screening, output validation, redaction, OPA policy, human approval, prompt variants
9	Observability, tracing, provenance	**Covered**	OpenTelemetry trace from webhook to agents, append-only audit trail with automatic checks, versioned Blackboard history, provenance guide
10	End-to-end evaluation, cost, box plot, dataset	**Not yet**	Per-agent accuracy evaluations complete; 40-run cost and latency study planned
11	Cloud deployment	**Not yet**	Complete Docker Compose stack of 16 services; cloud deployment planned

3. Criterion 1: Messaging architecture and context sharing

COVERED	**Requirement.** *Intermediate output from one agent must feed into another. A Blackboard (shared state) architecture is mandatory; other messaging architectures should be tried and compared, for example Blackboard against peer-to-peer.*

What we built

The shared context of every incident lives on a **Blackboard** implemented in `common/blackboard.py`. It is the only place where findings are stored, and every component that needs another component's output reads it from there.

How the Blackboard works

- **Append-only and versioned.** Each incident has its own history file (`data/blackboard/<incident_id>/history.jsonl`). A post never overwrites: it appends a new version for that author, section and key, flushing to disk before returning.
- **Seven sections.** `context` (the incident event, written by the backend), `hints` (early signals from agents), `evidence` (each agent's final findings), `synthesis` and `policy` (the Coordinator), `approval` (the approval service or a human), and `validation` (Jenkins re-run results).
- **Typed hints.** Early findings are structured as Pydantic hint models: failure window, failed stage, error signature, suspect files, anomaly summary, changed paths and guardrail flags.
- **Write permissions per author.** `_check_permissions` allows an agent to write only to `evidence/<its own name>` and `hints/<its own name>`; only the Coordinator may write `synthesis` and `policy`; only the approval service may write `approval`; only the backend may write `context`. An agent therefore cannot write, alter or forge another agent's conclusion.
- **Integrity and safety.** Every payload is redacted before it is stored, capped at 256 KB, and hashed. Posting an identical payload again returns the existing version, which makes retries safe. Each entry also records the `trace_id` and `span_id` of the span it was written under.
- **Reads.** `read` returns the latest or a specific version, `read_all` returns every entry for an incident, `history` returns the full ordered log, and `wait_for` lets a component wait for an entry to appear.
- **Compatibility.** `EvidenceStore` and `FileEvidenceStore` expose the original store interface on top of the Blackboard's `evidence` section, so agents and the Coordinator share one API.

How intermediate output feeds the next agent

The Coordinator reads the three agents' evidence documents with `read_all(incident_id)` and builds its report from them. The synthesis it produces is written back to the `synthesis` section, where the remediation engine and the web console read it. A fix proposal then flows through `policy`, `approval` and `validation` entries in the same way, and a failed validation makes the agents re-investigate with the earlier versions of the evidence still available for comparison. Each stage of the pipeline consumes the previous stage's output from the Blackboard rather than from a direct call.

Messaging architectures tried

The Blackboard is mandatory, and around it we implemented and can switch between two other messaging styles:

Style	Implementation	Carries
Event bus (RabbitMQ)	Durable topic exchange `incidents`; routing keys `incident.created`, `incident.reinvestigate`, `approval.requested`, `approval.decided`, `validation.completed`; each agent has its own durable queue	Notifications that something happened
A2A tasks	The Coordinator sends a task to each agent's `/tasks` endpoint; agents reply with an `evidence.ready` notification	The incident identifier only, never findings
Blackboard	`common/blackboard.py`	All findings, hints, synthesis, policy, approval and validation data

The `TRIGGER_MODE` setting (`a2a` or `rabbitmq`) selects which path starts an investigation in each agent, and a test (`tests/test_trigger_mode.py`) checks that no incident can be started by both paths. This provides the event-driven and the request-style messaging architectures side by side, with the Blackboard as the shared state in both.

Why a Blackboard rather than peer-to-peer

The design decision is recorded in `docs/decisions.md`. Because agents only notify and never hand over conclusions, they remain independent: three agents reading three different sources can corroborate or contradict each other, which is what the Coordinator weighs. Peer-to-peer exchange of conclusions would let one agent's mistake propagate into the others and would hide who concluded what. With a Blackboard every intermediate finding is versioned, attributed and available for audit.

Where to find it and how it is verified

Item	Location
Blackboard implementation	`common/blackboard.py`
Evidence store interface	`common/evidence_store.py`
Event publisher and consumers	`backend/events/publisher.py`, each agent's consumer module
Trigger mode switch	`agents/*/__main__.py`, `coordinator/app.py`
Tests	`tests/blackboard/test_blackboard.py`, `tests/common/test_evidence_store.py`, `tests/test_trigger_mode.py`

4. Criterion 2: Tool integration using MCP

COVERED	**Requirement.** *At least two external tools (library or custom) must be used, for example a search API, a database lookup, a file parser, a calculator, machine learning or forecasting.*

What we built

Agents reach their tools through servers that speak the Model Context Protocol, under `mcp_servers/`, built on the official `mcp` SDK (FastMCP) over streamable HTTP. A project rule in `docs/decisions.md` states that agents never import tool code directly.

The MCP servers

Server	Tools	Backing service	Deployment
Jenkins MCP	`get_build_summary`, `get_stage_summary`, `get_error_blocks`, `get_log_range`, `get_test_report`, `get_build_history`, `compare_with_last_success`	Jenkins REST API with API token from the environment	Compose service `mcp-jenkins`, port 8081, health check
Analytics MCP	`compute_signature` (deterministic SHA-256 signature of the failed stage and error blocks), `check_flakiness` (flakiness analysis of a named test over recent runs), `find_related_incidents` (earlier incidents with the same signature, with recurrence, cross-branch and regression flags)	Jenkins build history for flakiness; the Blackboard for related incidents	Compose service `mcp-analytics`, port 8082, health check
Prometheus MCP	`list_catalog`, `prometheus_query`, `prometheus_query_range`	Prometheus through the typed `PrometheusTool`	`agents/metrics_agent/mcp_server.py`

How agents use them

- **Client adapters.** `common/mcp_client.py` wraps each remote MCP tool as a CrewAI tool. The Jenkins agent receives the Jenkins and analytics tools through `get_jenkins_mcp_tools` and `get_analytics_mcp_tools`; server addresses come from `MCP_JENKINS_URL` and `MCP_ANALYTICS_URL`.
- **Bounded calls.** Every tool call has an explicit timeout (10 seconds by default) and a bounded number of retries (two). Transient failures are retried; others are reported to the agent.
- **Safe results.** A tool result is redacted and screened for prompt injection and toxic text, and wrapped as untrusted data, before the model reads it. Tool output is treated as untrusted input.
- **Read-only.** The Jenkins tools only read build data. No MCP tool can change Jenkins or the repository.

Other external tools

Beyond MCP, the system integrates further external tools through typed, timeout-bounded clients: Prometheus (range queries with bounded retry and typed errors), the GitHub API (commits, pull requests, diffs, branches and draft pull requests), OPA (policy evaluation over HTTP), the Gemini API

(through the shared LLM client with rate limiting and retries), and Slack, Teams and email for approval notifications. Together with the MCP servers this goes well beyond the minimum of two tools.

Where to find it and how it is verified

Item	Location
Jenkins and Analytics MCP servers	`mcp_servers/jenkins_mcp/server.py`, `mcp_servers/analytics_mcp/server.py`
Prometheus MCP server	`agents/metrics_agent/mcp_server.py`
Client adapters	`common/mcp_client.py`
Compose services	`docker-compose.yml` (`mcp-jenkins`, `mcp-analytics`)
Tests	`tests/test_mcp_servers.py`, `tests/metrics_agent/test_mcp_server.py`, `tests/jenkins_agent/test_history.py`

5. Criterion 3: Structured output with Pydantic

COVERED	**Requirement.** *The final output must be generated in a structured form (Markdown, table or JSON).*

What we built

Every piece of data that crosses a boundary in the system, between processes, services, queues, storage and agents, is a Pydantic v2 model. The final outputs are structured JSON for machines and Markdown for reviewers.

The shared evidence contract

Each agent writes one `Evidence` document per incident, defined in `common/models.py`. Its fields include the agent name, status (`completed`, `failed` or `insufficient_evidence`), a failure type from the fixed 11-class taxonomy, a confidence between 0 and 1, ranked root-cause hypotheses (each with supporting and contradicting evidence identifiers), evidence items (log excerpts, metrics, commits, test results with source and location), tool-call records, recommended next steps, an investigation-coverage score and recurrence information. Validators enforce the rules: confidence ranges, minimum text lengths, and timezone-aware UTC timestamps.

The canonical JSON Schema (`common/evidence_schema.json`) is generated from the model by `scripts/generate_evidence_schema.py` and never edited by hand. A test fails if the schema drifts from the model, if the taxonomy diverges, or if the wire version changes without the agents being updated together.

Structured output from the LLM

The Jenkins agent's model answer must parse into an `LLMAnswer` model that forbids extra fields and rejects an `unknown` classification asserted with confidence above 0.4. The shared LLM client requests structured output and, if parsing or validation fails, makes one repair call with the validation error before giving up. A CrewAI task guardrail validates the draft evidence before it is finalised.

The final incident report

The Coordinator's output is an `IncidentReport` (`coordinator/synthesis.py`): status (`resolved_cause`, `insufficient_evidence` or `conflicting`), failure type, confidence, a summary, each agent's finding, agreeing, abstaining and missing agents, per-agent dispatch outcomes, conflicts, hypotheses, recommended next steps and recurrence links. The same object renders to **Markdown** for a human reviewer (`to_markdown`) and serialises to **JSON** for the Blackboard, the API and the web console. An advisory `FusedReport` (`coordinator/report.py`) has its own Markdown renderer.

Other structured artefacts

- Blackboard entries and typed hint models; audit records; A2A task requests and agent cards.
- Fix plans from the remediation engine, policy verdicts, approval decisions, and validation results.
- Tabular views in the web console and Grafana dashboards built from these records.

Where to find it and how it is verified

Item	Location
Evidence models and taxonomy	`common/models.py`
Canonical schema and generator	`common/evidence_schema.json`, `scripts/generate_evidence_schema.py` (`make schema`)
LLM answer model and repair	`agents/jenkins_agent/agent.py`, `common/llm_client.py`
Incident report	`coordinator/synthesis.py`, `coordinator/report.py`
Tests	`tests/common/test_evidence_schema.py`, `tests/coordinator/test_synthesis.py`, `tests/common/test_llm_client.py`

6. Criterion 4: Agentic design patterns

COVERED	**Requirement.** *Implement suitable agentic design patterns and compare them with others: hierarchical, parallel and sequential, each with or without a worker node.*

What we built

The system combines three multi-agent patterns, each used where it fits, plus two reasoning patterns inside the Jenkins agent.

Hierarchical: a Coordinator over worker agents

The Coordinator, a Google ADK agent, supervises the three investigation agents as workers. It dispatches their tasks, collects their evidence, synthesises a single report per incident and attempt, and drives the remediation gate. Synthesis is idempotent, so a redelivered incident does not create a second report, and later changes are stored as numbered report revisions. After a restart the Coordinator recovers reports from persisted state.

Parallel: concurrent investigation

`coordinator/app.py` dispatches all three agents at once with `asyncio.gather`. Each agent run is timed and ends in one recorded outcome: completed, failed, timed out or not dispatched. A slow or failing agent never blocks the others, and a watcher marks an agent as timed out after a deadline. If a slow agent finishes later, the report is revised rather than lost. The Coordinator's completeness logic treats a missing or failed agent as an incomplete report instead of pretending it agreed.

Sequential: ordered steps inside each worker

Agent	Sequence
Jenkins agent	CrewAI Crew with a sequential process: fetch context, reason in a tool-calling loop (budgeted at 8 steps), finalise the evidence
Metrics agent	CrewAI Flow: build the time window, plan queries, query Prometheus, sanity-check, detect anomalies, correlate, score confidence, write evidence
Code agent	CrewAI Flow (`CodeInvestigationFlow`): fetch context, analyse changed files, reason about causes, finalise evidence
Remediation pipeline	Report, minimal patch, remediation policy, named human approval, pull-request policy, draft PR, Jenkins re-validation

Feedback loop

When Jenkins re-validation of an approved fix fails, the system publishes `incident.reinvestigate`. The agents re-run with the next attempt number, and the loop is bounded by a maximum number of attempts, after which the incident is escalated to a human.

Reasoning patterns for the Jenkins agent

The Jenkins agent supports interchangeable prompt variants, chosen with the `PROMPT_VARIANT` setting and defined as files under `agents/jenkins_agent/prompts/`:

Variant	Style
`react`	Think, call a read-only tool, observe, repeat (default)
`plan_execute`	Reasoning enabled: formulate an investigation plan first, then execute tools against it
`persona_junior_dev`, `persona_terse_sre`	Same task and safety rules, with a different role, goal and voice
`persona_adversarial`	A sceptical CI auditor who rejects claims that are not grounded in cited evidence

Comparison with alternatives

Two comparisons are recorded in the repository. First, the Metrics agent's structured, tool-using pipeline was evaluated against two alternatives: a single fixed-threshold rule and a single LLM call given the same data (Section 14). Second, two fusion strategies for the Coordinator, noisy-OR and a weighted mean, were compared on labelled scenarios, including a monotonicity check over 729 combinations (`docs/fusion_method_comparison.md`). The messaging comparison between A2A and RabbitMQ triggering is supported by the `TRIGGER_MODE` switch described in Section 3.

Where to find it

Item	Location
Parallel dispatch, outcomes, revisions	`coordinator/app.py`, `coordinator/revisions.py`
Hierarchical Coordinator	`coordinator/agent.py`, `coordinator/service.py`
Flows and Crew	`agents/code_agent/agent.py`, `agents/metrics_agent/`, `agents/jenkins_agent/agent.py`
Prompt variants	`agents/jenkins_agent/prompts/*.yaml`
Tests	`tests/coordinator/test_dispatch_all.py`, `test_dispatch_synthesis.py`, `test_revisions.py`

7. Criterion 5: Task monitoring and logging with callbacks

COVERED	**Requirement.** *Agents must include a monitoring mechanism that tracks intermediate outputs, execution flows and errors for transparency and debugging. Can a heat map be created within each agent of the multi-agent system?*

What we built

Monitoring works at three levels: callbacks inside the agent, an audit trail of every run, and dashboards that aggregate both.

Callbacks inside CrewAI agents

`common/monitoring.py` provides `create_step_callback` and `create_task_callback`, which are attached to the Jenkins agent's CrewAI `Agent` and `Task`. They fire on every reasoning step, tool call and task completion and record a structured event with the incident, agent, run identifier, prompt variant, step index, tool, a hash of the arguments, duration, success flag, error and a short redacted preview. The recorder never raises, so a monitoring fault cannot break an investigation.

The same module exports Prometheus metrics: `agent_steps_total`, `agent_errors_total`, `agent_tool_calls_total` (by tool), `agent_step_duration_seconds` (a histogram) and `llm_tokens_total`, each labelled by agent and prompt variant.

Audit trail for every agent

The Metrics and Code agents, which are deterministic flows, are monitored through the shared audit layer (`common/audit.py`). `agent_run` records `agent_started` and a terminal `agent_completed` or `agent_failed` event, and each agent adds `tool_call`, `hypothesis_formed` and `evidence_written` events. The Coordinator adds an `agent_dispatch` event with the outcome and duration of every agent it asked. Records are stored in PostgreSQL (`audit_events`) or as JSON lines, and a failure of the audit log degrades traceability without failing the investigation.

Dashboards and heat map

- **Agent Ops dashboard** (20 panels): a run-duration **heat map** by agent, average run duration over time, tool-call latency by agent, an investigation-time table, time to report, approval rate, fix success rate, verdict and approval breakdowns.
- **Approvals and Operations dashboard** (18 panels): approvals, outcomes and the incident lifecycle.
- **Alert rules** (4): approval waiting over an hour, agents failing or timing out, an incident escalated to a human, and an incident with no report after ten minutes.

The heat map is built from the audit events of all agents, so each agent has its own row and the colour shows how run time is distributed over the day. Intermediate outputs are also visible in the Blackboard's versioned history.

Where to find it and how it is verified

Item	Location
Callbacks and Prometheus metrics	`common/monitoring.py`
Audit layer	`common/audit.py`, `agents/metrics_agent/audit.py`, `migrations/versions/`
Dashboards	`monitoring/grafana/dashboards/agent_ops.json`, `ops.json`; generator `scripts/generate_agent_ops_dashboard.py`
Alert rules	`monitoring/grafana/provisioning/alerting/rules.yml`
Tests	`tests/common/test_audit.py`, `tests/metrics_agent/test_audit.py`

8. Criterion 6: Agent-to-agent communication with A2A

COVERED	**Requirement.** *Agents must communicate using the A2A protocol, with interoperability between frameworks, for example one agent defined with ADK and another with CrewAI.*

What we built

The Coordinator is a Google ADK agent and the three investigators are CrewAI agents. They talk to each other only over A2A, using the official `a2a-sdk`. This is the interoperability the criterion asks for: two different agent frameworks cooperating through one open protocol.

The protocol layer

- **Contract.** `common/a2a_contract.py` defines the agent cards and the message kinds. The one notification kind, `evidence.ready`, carries only the incident identifier and the agent name.
- **Agent cards.** Each agent publishes a card describing its skills and their input and output schemas. For example the Jenkins agent's card (`a2a/jenkins_agent.json`) offers `investigate_incident` and `reinvestigate_incident`.
- **Task server.** `common/a2a_server.py` turns a CrewAI agent into an A2A service: it serves the card, a health endpoint and a `POST /tasks` endpoint that accepts an `AgentTaskRequest` (incident, attempt, and for re-investigation the previous failure summary and new build URL).
- **Security.** Requests need a shared bearer token, compared in constant time; the service refuses to run without a configured token.
- **Idempotency.** Executions are cached per incident and attempt and guarded by locks, so a repeated task does not run twice.
- **Trace context.** The Coordinator adds the W3C `traceparent` header to each task; the task server extracts it and runs the task inside an `a2a_execute_task` span, so the agent's work joins the incident's trace.

The two directions of communication

Direction	What happens	Where
Coordinator to agent	Sends a task to the agent's `/tasks` endpoint with timeouts and error handling; the outcome is recorded	`coordinator/app.py`
Agent to Coordinator	After writing evidence to the Blackboard, the agent sends `evidence.ready` through an A2A client	`coordinator/a2a_client.py`

Keeping the Blackboard intact

The A2A messages deliberately carry no findings. A receiver that wants the evidence reads it from the Blackboard. This keeps the Blackboard authoritative and guarantees that no agent can hand another agent a conclusion.

Interoperability in practice

Because CrewAI and ADK pin conflicting versions of shared libraries (OpenTelemetry in particular), each service runs in its own container with its own requirements. A2A is the only boundary between them, which demonstrates that the frameworks interoperate without sharing a Python environment. A feasibility spike that preceded the implementation is documented in `docs/a2a_spike_findings.md`.

Where to find it and how it is verified

Item	Location
Contract and cards	`common/a2a_contract.py`, `a2a/jenkins_agent.json`
Task server	`common/a2a_server.py`
Coordinator side	`coordinator/app.py`, `coordinator/a2a_client.py`, `coordinator/agent.py`
Tests	`tests/test_a2a_server.py`, `tests/coordinator/test_a2a_contract.py`, `test_dispatch.py`

9. Criterion 7: Agentic frameworks: CrewAI and ADK only

COVERED	**Requirement.** *Both CrewAI and ADK must be used as the agentic framework. No other agentic framework may be used.*

What we built

Component	Framework	How it is used
Jenkins agent	CrewAI	A `Crew` with one `Agent` and one `Task` on a sequential process; tools, step budget, callbacks and a task guardrail attached
Code agent	CrewAI	A `Flow` whose steps are decorated with `@start` and `@listen`
Metrics agent	CrewAI	A `Flow` wrapping the investigation pipeline
Coordinator	Google ADK	A `BaseAgent` that receives A2A notifications, collects evidence from the Blackboard and synthesises it

Decision and enforcement

The framework decision is recorded as item 1 in `docs/decisions.md`: CrewAI for the three investigation agents, Google ADK for the Coordinator, and no other agentic framework permitted. The earlier prototype used LangGraph; it was replaced by CrewAI in the migration that also introduced the ADK Coordinator and A2A. That migration kept behaviour identical: the same evidence documents, classifications, confidence values and audit records. A search over `agents/`, `coordinator/`, `common/`, `backend/` and `mcp_servers/` finds no LangChain or LangGraph imports, and the shared LLM client no longer depends on LangChain.

Reaching the model

CrewAI reaches Gemini through an adapter (`common/crew_llm.py`, a `BaseLLM` subclass over the shared `LLMClient`), so the rate limiting, retries, redaction and structured-output repair of the shared client apply to CrewAI as well. ADK uses Gemini natively.

Why each framework fits

- CrewAI's role, task and tool model suits the Jenkins agent's tool-calling investigation, and its Flow primitive gives the deterministic agents an explicit, ordered, observable step structure.
- ADK's agent and event model, together with its multi-agent support, suits a supervising agent that collects and synthesises.

Where to find it

Item	Location
Decision record	`docs/decisions.md` (items 1, 2, 8 and the framework boundary note)
Dependency pins	`agents/*/requirements.txt` (crewai), `coordinator/requirements.txt` (google-adk), `requirements-dev.txt`
LLM adapter	`common/crew_llm.py`, `common/llm_client.py`

10. Criterion 8: Guardrails and security

COVERED	**Requirement.** *Secure the agentic system with guardrails and callbacks for data validation, such as type checking of tool returns and screening user prompts for toxic language or injection. Use multiple prompts that vary in structure or content, for example personas with abusive language.*

What we built

Security is layered so that no single check is relied on. Untrusted text is screened on the way in, model output is validated on the way out, secrets are redacted everywhere, and actions are gated by policy and a human.

Layer 1: screening untrusted input

Build logs, commit messages, pull-request text and tool results are untrusted. `screen_untrusted_text` in `common/guardrails.py` redacts secrets, then checks the text against:

- **Prompt-injection patterns**, such as instructions to ignore previous instructions, disregard prior prompts, override the system prompt, enter developer mode, mark the build as success, disable tests or disable security checks.
- **Toxic and abusive language patterns.**

Matches are replaced with `[FLAGGED_PROMPT_INJECTION]` or `[FLAGGED_TOXIC_LANGUAGE]`, and the whole text is wrapped in `<untrusted_data>` tags so the model treats it as data. Evidence is never silently dropped. A flag is also posted to the Blackboard as a hint, so the event is visible downstream. The MCP client applies the same screening to every tool result.

Layer 2: validating output

- **Task guardrail.** `evidence_guardrail` runs on the Jenkins agent's draft evidence with one retry. It rejects evidence that contains an unredacted secret, recommends disabling, skipping, deleting or removing tests, bypassing security or disabling OPA, or carries a confidence outside 0 to 1.
- **Type checking of tool returns.** `validate_tool_output` validates a tool result against a Pydantic model before use.
- **Schema validation.** The LLM answer must parse into a strict model, `unknown` is capped at 0.4 confidence, and the shared client repairs malformed structured output once.

Layer 3: redaction

`common/redaction.py` masks bearer tokens, credentials in URL query strings, provider API keys and values under secret-looking keys, recursively. It is applied before text reaches an LLM, the Blackboard, the evidence store or the audit trail, and again as a check at write time.

Layer 4: policy and human control

- **OPA remediation policy** (`policies/remediation.rego`) blocks a proposed fix with confidence below 0.5, more than 5 files or 40 lines, any protected path (pipeline, CI, monitoring, migrations, Dockerfiles and the policies themselves), any secret path, anything outside the application under test, deleting a test, or adding a skip or disabled check.

- **Pull-request policy** requires a recorded approval with a named reviewer, a draft pull request and auto-merge off.
- **Fail closed.** If OPA cannot be reached the proposal is blocked.
- **Human approval.** A person decides before a branch or pull request exists.

Layer 5: service security

The A2A endpoints and the approval API require bearer tokens compared in constant time. The Jenkins webhook checks a shared secret and enforces a 64 KB body limit. Approval links are HMAC-signed, the approval page uses CSRF tokens, and the web console enforces viewer, reviewer and admin roles.

Prompt variation and adversarial personas

The Jenkins agent runs under five prompt variants that differ in structure (ReAct against plan-and-execute) and in content (junior developer, terse SRE, adversarial auditor personas). `tests/security/personas.yaml` holds persona scenarios in the voices of different people: an abusive developer commit message, an angry reviewer comment, a console log carrying an injected instruction to report success, a polite senior SRE, a terse bot message, a confused junior developer, and a log containing a leaked token. `test_personas_yaml_scenarios` checks that the abusive and injected inputs are flagged, that the polite and neutral ones are not, and that the token is redacted.

Where to find it and how it is verified

Item	Location
Guardrails	`common/guardrails.py`, `common/redaction.py`, `common/mcp_client.py`
Policies	`policies/remediation.rego`, `policies/pull_request.rego`, `coordinator/policy.py`, `coordinator/remediation_gate.py`
Tests	`tests/security/test_guardrails.py`, `tests/security/personas.yaml`, Rego tests (`policies/*_test.rego`), `tests/coordinator/test_policy.py`, `test_remediation_gate.py`

11. Criterion 9: Observability: tracing, auditability and provenance

COVERED	**Requirement.** *Implement a suitable observability mechanism for tracing, auditability and provenance.*

What we built

Observability has three parts: distributed tracing that follows an incident across services, an append-only audit trail with automatic consistency checks, and a provenance path from any claim back to the raw evidence.

Distributed tracing

Every incident has one OpenTelemetry trace (`common/tracing.py`). Each service installs its tracer provider at start-up, with an OTLP exporter when `OTEL_EXPORTER_OTLP_ENDPOINT` is set and a console exporter when `OTEL_TRACES_CONSOLE=1`.

Span	Started by	Purpose
`webhook.create_incident`	The Jenkins webhook handler	Root span; records the job, build number, incident id and whether the incident was new
`coordinator.handle_message`	The Coordinator, on a RabbitMQ message	Continues the trace from the message's `traceparent` header
`coordinator.run_agent`	The Coordinator, once per agent	Records the agent and its outcome
`coordinator.synthesize`	The Coordinator	The fusion step
`a2a_execute_task`	Each agent's A2A task server	Continues the trace from the `traceparent` header on the Coordinator's call

The trace travels in three hops. The backend publishes `incident.created` with the trace as a message header. The Coordinator reads it, continues the trace, and adds the same header to every A2A call. Each agent's task server extracts it and runs its task inside the span. Every Blackboard entry written inside a span records its `trace_id` and `span_id`, including the incident's `context/incident_event` entry, so an incident can be matched to its trace from the Blackboard.

Audit trail

The audit trail records what happened while an incident was handled, as append-only records. A later outcome is a later record, never an update, and each record redacts its own summary and payload.

Stage	Events recorded
Incident	`incident_created`, `incident_status_changed`
Agents	`agent_started`, `tool_call`, `hypothesis_formed`, `evidence_written`, `agent_completed` or `agent_failed`
Coordinator	`agent_dispatch` (outcome and duration per agent), `synthesis_completed`, `synthesis_revised`
Remediation	`remediation_proposed`, `policy_evaluated`, `approval_requested`, `approval_decided`, `pull_request_created`, `revalidation_completed`

Every incident status change goes through `backend/incident_lifecycle.py`, which writes the audit row in the same database transaction as the status. How an incident ended (resolved by a fix, rejected, escalated, or still in progress) is derived from the audit events rather than stored separately (`monitoring/sql/incident_outcomes.sql`).

Automatic consistency checks

`scripts/check_audit.py` verifies a trail against a set of rules in `common/audit_check.py`. Errors include an agent start without an end, duplicate or out-of-order report revisions, an unredacted record, a pull request without a prior approval, an approval over a policy denial without a recorded override, a final status the events do not justify, and a broken status chain.

Provenance

`docs/provenance.md` describes how to trace any conclusion back to its source. A hypothesis in the report cites evidence identifiers, which are namespaced by agent (for example `code_agent:file-1`). The identifier leads to an evidence item on the Blackboard, which records its source, file and line. The Jenkins MCP `get_log_range` tool then fetches the exact console lines, and the `trace_id` on the Blackboard entry leads to the incident's trace. The Blackboard's versioned, hashed history gives the provenance of every intermediate finding.

Where to find it and how it is verified

Item	Location
Tracing	`common/tracing.py`, `backend/routes/webhooks.py`, `backend/events/publisher.py`, `coordinator/app.py`, `common/a2a_server.py`, `common/blackboard.py`
Audit trail and checks	`common/audit.py`, `common/audit_check.py`, `scripts/check_audit.py`, `backend/incident_lifecycle.py`
Documents	`docs/provenance.md`, `docs/audit_workflow.md`
Tests	`tests/test_tracing.py`, `tests/common/test_audit.py`, `tests/common/test_audit_check.py`, `tests/test_incident_lifecycle.py`

12. Criterion 10: End-to-end evaluation, cost and running time

NOT YET	**Requirement.** *Perform end-to-end performance evaluation with cost and running-time estimates over multiple runs (about 40), with variance in prompts (different architectures such as ReAct, different persona content). Produce a box plot and build a dataset against which performance is reported.*

What has been completed

Accuracy evaluations are complete at the level of each agent and of the Coordinator, using ground-truth scenarios (Section 14). The groundwork for the end-to-end study is also in place:

- **Dataset seed.** 31 scenario files in `scenarios/` define incidents and expected outcomes: 13 for the Coordinator, 14 for the Metrics agent and 4 for the Code agent, plus recorded build fixtures for the Jenkins agent.
- **Prompt variance.** Five prompt variants (ReAct, plan-and-execute, three personas) are selectable by one setting.
- **Trigger variance.** `TRIGGER_MODE` switches between A2A and RabbitMQ triggering.
- **Measurement hooks.** Every agent run records its duration in the audit trail, the Agent Ops dashboard aggregates run time by agent, and the monitoring module counts LLM tokens per agent and variant. `config/pricing.yaml` holds per-model token prices for the cost estimate.

What remains and the plan

1. Build a harness that runs each scenario about 40 times in total across prompt variants and trigger modes, storing wall-clock time, token counts and estimated cost for every run.
2. Write the results to a documented dataset and report the distributions as box plots per variant.
3. Pace the LLM runs to the provider's quota. The free tier of the Gemini API allows roughly 20 requests per model per day in our observation, so the LLM runs are spread over several days, with answers cached for reproducibility. The deterministic agents can be run in bulk without this limit.

Status. This criterion is scheduled for the final week of the plan. The measurement infrastructure it needs already exists.

13. Criterion 11: Cloud deployment

NOT YET	**Requirement.** *Deploy the project on a cloud platform.*

What has been completed

The system is fully containerised and runs as a single Docker Compose stack of 16 services, which is the basis for any cloud deployment:

Group	Services
Messaging and data	`rabbitmq`, `postgres`
Pipeline	`jenkins`, `target-app`
Agents and coordination	`jenkins-agent`, `metrics-agent`, `code-agent`, `coordinator`, `agent-api`, `opa`
Tools	`mcp-jenkins`, `mcp-analytics`
Monitoring and console	`prometheus`, `grafana`, `ui`, `mailhog` (test mail catcher)

Every service has its own Dockerfile, health checks where it matters, dependency ordering (`depends_on` with health conditions), environment-driven configuration validated at start-up, named volumes for state, and no secrets in the source tree.

What remains and the plan

4. Select the target platform and build and push the service images to a registry.
5. Provide PostgreSQL and the secrets through the platform's managed services rather than local files.
6. Expose the web console and the webhook endpoint over HTTPS, and point the Jenkins webhook at the deployed backend.
7. Run the end-to-end scenario on the deployed stack and record the result.

Status. Cloud deployment is scheduled for the final week of the plan.

14. Evaluation results

This section reports the quantitative evaluations completed so far. They are scenario-based: each scenario has a known correct answer, so a verdict can be scored.

14.1 Metrics agent

The Metrics agent was evaluated by `scripts/score_metrics_agent.py` against two alternatives over the same data: a fixed-threshold alert (latency over 1 second, CPU over 0.8 cores, memory over 512 MiB, or any availability drop) and Gemini 3.5 Flash given the same time series as text (one run, temperature 0).

System (14 scenarios)	Accuracy	Fault recall (9 faults)	False-positive rate (5 no-cause cases)
Metrics agent	100%	9 of 9	0 of 5
Threshold alert	71%	6 of 9	1 of 5
Gemini 3.5 Flash	64%	4 of 9	0 of 5

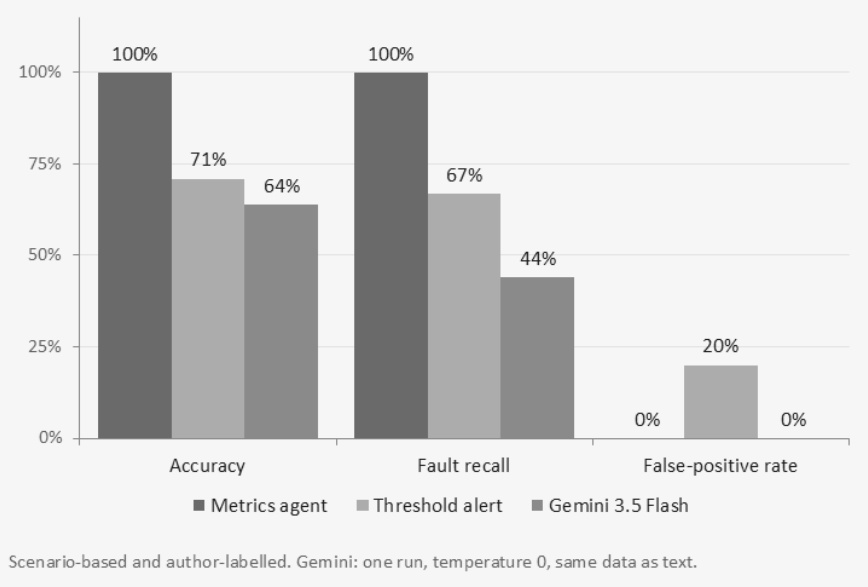


Figure 2. Scenario benchmark: Metrics agent, threshold alert and Gemini 3.5 Flash.

A severity sweep of 26 generated cases varies fault size and noise. The agent detected 14 of 21 faults with the correct type against 4 of 21 for the threshold alert, and neither raised a false alarm on the five healthy runs. The agent's undetected faults are those smaller than its minimum-effect floors (latency steps up to 0.25 s, CPU up to 0.03 cores, memory growth under 5 MiB). Gemini named the right type on the four latency-type cases but missed the CPU burn, the combined latency-and-CPU case, the memory leak, the partial-data case and the outage, calling the CPU burn a test failure and the outage a deployment failure. Its average confidence was 0.93 when right and 0.88 when wrong.

14.2 Jenkins agent

The Jenkins agent's rule-based classifier correctly classified all 6 ground-truth builds on its own, without the LLM: a failing unit test, a syntax error, a missing-module import, twenty simultaneous test failures, a rejected git checkout (authentication) and an unreachable git checkout (network). Guard-rail tests

verify that the LLM cannot relabel a confirmed test failure as an environment problem, that a truncated log lowers confidence, and that only the top five failures are sent to the model.

14.3 Code agent

Four scenarios, TC-03 (dependency regression), TC-08 (configuration error), TC-12 (anomaly with no code change) and TC-14 (no clear root cause), meet every stated criterion, including the two abstention cases in which the agent must not blame an innocent commit.

14.4 Coordinator

Thirteen coordinator scenarios cover agreement, conflict, missing and failed agents, partial reporting and abstention. A comparison of two fusion methods on ten labelled scenarios gave noisy-OR 10 of 10 top-ranked correct against 9 of 10 for a weighted mean, and 0 monotonicity violations over 729 cases against 324 (`docs/fusion_method_comparison.md`).

> **Method.** Scenarios and labels were written by the team, so the sweep and the live runs provide the evidence that is independent of the detector's design.

15. Project timeline and progress against the plan

The synopsis set an eight-week plan. The table lists each week's planned deliverables and what was delivered, with the dates of the corresponding commits in the repository.

Week	Planned (synopsis)	Delivered	Dates	Status
1	Webhook, Jenkins pipeline, agent skeletons, Prometheus and Grafana, shared evidence schema, audit store, OPA skeleton	All of these	30 Sep - 2 Oct	**Done**
2	Agent v1 for Jenkins, Code and Metrics; evidence and audit layer; mock incident end to end	All three agents write evidence to the shared store	5 Oct	**Done**
3	Log, configuration, latency and CPU scenarios; code-side reasoning; policy rules and audit logging	Scenario suites for each agent; policy rules; audit trail	5 - 6 Oct	**Done**
4	Confidence scoring, Jenkins report sections, anomaly panels, Coordinator synthesis	Confidence models, Coordinator synthesis and incident report, fusion	25 Oct	**Done**
5	Hardening; minimal patch generation; fix plans; git branching workflow	Remediation engine and branch workflow	25 Oct	**Done**
6	OPA integration, draft-PR workflow, re-validation trigger, Agent Ops dashboard	Policy-gated draft PRs, validation webhook, Agent Ops dashboard	25 - 26 Oct	**Done**
7	Re-validation feedback loop, approval and notification integration, remediation audit events, operations dashboards	Approval API, notifier and signed links, lifecycle audit, operations dashboard, alerts	26 - 30 Oct	**Done**
8	Accuracy evaluation, documentation, integration tests, benchmark results, demo	Per-agent evaluations and tests done; benchmark at scale and cloud deployment remain	From 30 Oct	**In progress**

Seven of the eight planned weeks are delivered and the eighth has begun. Work delivered beyond the plan includes the web console with roles, flaky-test and recurrence detection, A2A and distributed tracing, and Grafana alert rules.

16. Verification and testing

The repository has an automated test suite that runs without network access or an API quota: external services are driven through mock transports, and the LLM through a scripted mock client. The full suite, run on 30 October 2026, reports **1433 passed and 13 skipped**. The skipped tests need an OPA binary or a PostgreSQL test database.

Area	What the tests cover
Evidence contract and shared code	Schema generation and drift, evidence store, LLM client retries and repair, redaction, audit layer and checks
Jenkins agent	Parsing, rule classifier, ground-truth scenarios, LLM guard-rails, MCP tool use, flakiness
Metrics agent	Window, sanity checks, detectors, correlation, confidence, 14 scenarios, audit, MCP server, evaluation script
Code agent	GitHub client, diff analysis, reasoning, scenarios, remediation engine
Coordinator	Synthesis, fusion, dispatch outcomes, revisions, A2A contract, policy client, remediation gate, trigger modes
Security	Injection and toxicity screening, redaction, evidence guardrails, persona scenarios, Rego policy tests
Backend and console	Webhook idempotency, approvals, notifications, validation, lifecycle audit, 81 console API tests

17. Conclusion and remaining work

JenkinsGuardian AI meets nine of the eleven project criteria. It shows a team of agents in two different frameworks, cooperating through an A2A boundary and a shared Blackboard, using tools over MCP, producing structured, auditable output, and acting only under policy and human control. Its evaluations show that structure around a model, in the form of statistics, tools, validation and abstention, outperformed a bare LLM on our metric scenarios.

Remaining work in the final week

Item	Plan
Criterion 10: end-to-end evaluation	Harness of about 40 runs across prompt variants and trigger modes; time, tokens and cost per run; box plots; documented dataset
Criterion 11: cloud deployment	Images to a registry; managed database and secrets; HTTPS endpoints; end-to-end run on the deployed stack
Week 8 deliverables	Benchmark results, documentation, integration tests and a final demo
Extensions	LLM query planner for the Metrics agent; LLM-assisted reasoning for the Code agent and the Coordinator narrative, each scored against the deterministic baselines already built

Appendix A. Repository map

Path	Contents	Criteria
common/	Blackboard, evidence models, audit, guardrails, redaction, tracing, monitoring, A2A contract and server, MCP client, LLM client	1, 3, 5, 6, 8, 9
agents/jenkins_agent/	CrewAI Crew, classifier, parsing, prompt variants	2, 4, 5, 7, 8
agents/metrics_agent/	CrewAI Flow, anomaly detection, correlation, confidence, Prometheus tool and MCP server	2, 4, 7
agents/code_agent/	CrewAI Flow, diff analysis, reasoning, remediation engine	4, 7
coordinator/	ADK Coordinator, dispatch, synthesis, fusion, revisions, policy client, remediation gate	3, 4, 6, 7
mcp_servers/	Jenkins and analytics MCP servers	2
policies/	OPA remediation and pull-request policies with tests	8
backend/	Webhook, events, approvals, notifications, lifecycle audit, console API	1, 8, 9
monitoring/	Prometheus and Grafana configuration, dashboards, alert rules, SQL	5, 9
scenarios/	Ground-truth scenarios for agents and the Coordinator	10
scripts/	Fault injection, evaluation, schema generation, audit check, dashboard generator	3, 9, 10
docs/	Decisions, provenance, audit workflow, A2A spike, fusion comparison, draft-PR workflow	all
tests/	Automated tests per area	all
docker-compose.yml	The 16-service stack	11

Appendix B. How to reproduce the results

The stack and the tests are driven from the repository root. A `.env` file supplies credentials; `.env.example` lists the variables.

Goal	Command
Start the stack	`docker compose up -d --build`
Run the full test suite	`python -m pytest tests`
Score the Metrics agent against the baselines	`python -m scripts.score_metrics_agent`
Add the Gemini baseline	`python -m scripts.score_metrics_agent --llm --llm-scope scenarios`
Inject a metrics fault and run an incident	`python scripts/inject_faults.py cpu` (also `slow`, `leak`, `healthy`)
Check an audit trail	`python scripts/check_audit.py --all`
Regenerate the evidence schema	`make schema`
Compare fusion methods	`python scripts/compare_fusion_methods.py --write`